NATIONAL ECONOMICS UNIVERSITY
FACULTY OF DATA SCIENCE AND ARTIFICIAL INTELLIGENCE

GROUP ASSIGNMENT REPORT

Midterm

Web Design and Programming ([Course code])

Rùa Toán: A Mathematics Practice Website

A source-attributed problem library for lower secondary students

Group: [Group name/number]
Class: [Class name]
Instructor: M.Sc. Le Duy Khanh
Semester: Semester 1, 2026–2027
GitHub: [GitHub-repository-URL]
Demo: https://ruatoan.vercel.app

Team Members

No.	Full name	Student ID	Role
1	[Full name]	[Student ID]	[Role and work]
2	[Full name]	[Student ID]	[Role and work]
3	[Full name]	[Student ID]	[Role and work]
4	[Full name]	[Student ID]	[Role and work]

Hanoi, November 2026

Declaration

We declare that this report describes the Rùa Toán website developed for Web Design and Programming. The group must verify this statement, complete the member and contribution information, and cite any external material or AI assistance used in the submitted version.

Task Allocation and Contributions

Member	Work done	Contribution (%)
Member 1	[Describe verified contribution]	[%]
Member 2	[Describe verified contribution]	[%]
Member 3	[Describe verified contribution]	[%]
Member 4	[Describe verified contribution]	[%]
	Total	**[100]**

Contribution percentages are pending confirmation from the group and must sum to 100%.

Table of Contents

1 Introduction . . . 4
1.1 Background and motivation . . . 4
1.2 Objectives . . . 4
1.3 Scope and target users . . . 4
1.4 Technologies . . . 4
1.5 Report structure . . . 4
2 Analysis and Design . . . 5
2.1 Requirements . . . 5
2.1.1 Functional requirements . . . 5
2.1.2 Non-functional requirements . . . 5
2.2 Interface design (UI/UX) . . . 5
2.3 System architecture . . . 6
2.4 Data design . . . 6
2.5 API design . . . 6
3 Implementation . . . 7
3.1 Project structure . . . 7
3.2 User interface and problem rendering . . . 7
3.3 Photo storage and API logic . . . 7
3.4 Content collection and source control . . . 8
3.5 Deployment . . . 8
4 Results and Conclusion . . . 8
4.1 Verification status . . . 8
4.2 Achievements and limitations . . . 8
4.3 Lessons learned and future work . . . 9
References . . . 10
A Appendix . . . 10
A.1 Setup and run instructions . . . 10
A.2 Demo accounts . . . 10
A.3 Use of AI tools (if any) . . . 10

List of Figures

1 Primary navigation and study flow in Rùa Toán. 6
2 High-level Rùa Toán architecture. 6

List of Tables

1 Technologies identified in the project source. 5
2 Photo-submission API routes implemented in the source. 6
3 Functional verification checklist; results have not yet been measured. 8

1. Introduction

1.1. Background and motivation

Students preparing for lower-secondary mathematics competitions often need to search across exam papers, topic collections, and reference materials. Rùa Toán is a Vietnamese website that brings these resources into one browsable library. It focuses on mathematics for grades 6–9, entrance examinations for grade 10 specialised schools, topic-based practice, magazines, and selected olympiad material.

The project is designed for students who want to find a problem, read its original statement with mathematical notation, work it out on paper, and keep a record of their attempt. It also provides teachers and parents with a structured place to browse exam collections and external learning resources. The site deliberately presents problem statements without generated hints or solutions; students can submit photographs of their own work instead.

1.2. Objectives

The project has the following objectives:

- Organise mathematics problems by topic, grade, exam type, year, and source.
- Let users browse complete exam collections and open individual problems.
- Render mathematical notation and provide readable layouts on desktop and mobile.
- Let a student submit, view, and delete photographs of written work without creating an account.
- Preserve attribution for exam papers and link books and documents to their source pages.

1.3. Scope and target users

The primary users are Vietnamese students in grades 6–9 preparing for gifted-student exams and grade 10 specialised-school entrance exams. Teachers and parents are secondary users. The site includes five topic areas: number theory, algebra, inequalities, geometry, and combinatorics. The wider collection also includes magazines, topic exercises, and olympiad papers.

The project provides browsing, filtering, problem pages, printable exam pages, local progress summaries, and photo submissions. It does not currently provide user accounts, automatic answer checking, a solution library, teacher dashboards, or cross-device progress synchronisation. Photo submissions are associated with a random browser-device identifier, not a verified identity.

1.4. Technologies

At the time this report was prepared, the JSON content files contained 3,381 collections and 33,929 problem records, together with 90 lessons, 186 book records, and 89 document records. These are repository counts, not independently verified measures of unique or educationally validated content.

1.5. Report structure

Section 2 describes the requirements, interface, architecture, data model, and API. Section 3 explains the implementation and deployment. Section 4 records the verification status, limitations, and future work.

Component	Technology	Purpose
Web framework	Next.js 16, React 19	Server-rendered pages, routing, metadata, and API routes
Application code	TypeScript, HTML, CSS	Page and interaction implementation
Content	Static JSON files	Problem collections, lessons, books, and documents
Mathematics	KaTeX	Render LaTeX expressions in problem statements
Submission storage	PostgreSQL	Store submitted image bytes and metadata
Content tooling	Python, Node.js	Crawl, transform, and validate source content
Deployment	Vercel	Host the Next.js application; project configuration selects the Singapore region

Table 1. Technologies identified in the project source.

2. Analysis and Design

2.1. Requirements

2.1.1. Functional requirements

FR1. Browse exam collections by kind and filter eligible collections by grade, year, province, and practice status.

FR2. Browse problems by topic, grade, kind, year, or a text query, and open a problem with its source attribution.

FR3. Read mathematical notation rendered from the stored statement and navigate between problems in the same collection.

FR4. Print a complete exam collection from its collection page.

FR5. Submit up to six photographs per request, view them, and delete them from the same browser device.

FR6. Browse topic lessons, books, and documents, with outbound links to source pages where available.

2.1.2. Non-functional requirements

The interface should be responsive, usable without an account, and clear about the source of each problem. Mathematical text should remain readable when printed. The application should avoid storing student names: submissions use a random browser identifier that is hashed on the server. The current implementation has per-image size checks and a per-device daily submission limit; these controls are application-level safeguards and are not an account or identity system.

2.2. Interface design (UI/UX)

The interface uses a dark green background with bright accent colours, a turtle mascot, and five topic categories. The main navigation links to problem practice, exam collections, topic material, books, and submitted work. On the home page, students can open a daily problem and see the size of the content library. The practice page presents problem cards; an individual problem page includes its statement, source, previous/next navigation, related problems, and a photo-submission control. An exam page presents the collection in order and offers a print action.

Figure 1 summarises the main student journey. It is a functional flow diagram, not a screenshot; screenshots of the running site should be inserted if the course requires visual evidence.

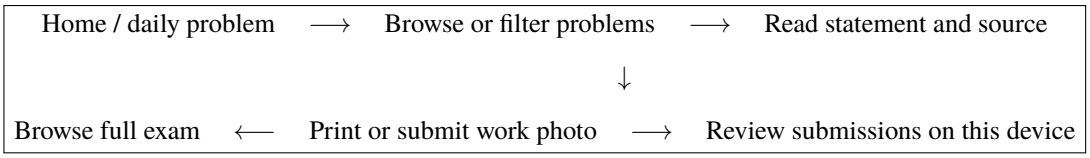


Figure 1. Primary navigation and study flow in Rùa Toán.

2.3. System architecture

The application is a Next.js App Router project [3] deployed through Vercel [4]. Most learning content is stored as JSON in the repository and read by server-side page code. The browser receives rendered pages and uses client-side components for navigation state, progress summaries, image compression, and photo management. The photo API uses PostgreSQL for image bytes and metadata, stored using PostgreSQL's binary data support [2]; this is the only identified application feature requiring a database. Figure 2 shows the separation between static learning content and submitted work.

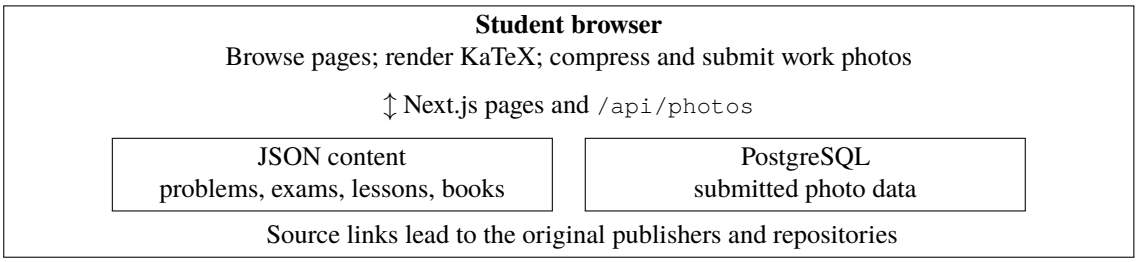


Figure 2. High-level Rùa Toán architecture.

2.4. Data design

The content model has two main records. A *Collection* stores its identifier, kind, title, grade, optional year and province, source name and URL, and an ordered list of problems. A *Problem* stores its identifier, label, statement, and one or more topic tags. Separate JSON files contain lessons, books, and documents. Topics written with older identifiers are mapped to the five public topic categories when read.

Submitted work is stored separately in the PostgreSQL schema `thcs_hard`. Each photo row contains a UUID, a hashed device identifier, problem metadata, MIME type, byte count, binary image data, and creation time. Indexes support listing photos by device and time or by problem. The browser keeps a summary in local storage; the database remains the source for photo records.

2.5. API design

Method	Endpoint	Purpose
GET	`/api/photos`	List up to 500 photo metadata records for the supplied device; optional problem filter
POST	`/api/photos`	Save up to six WebP/JPEG images for a problem; checks per-file size and daily device limit
GET	`/api/photos/{id}`	Retrieve an image by UUID
DELETE	`/api/photos/{id}`	Delete an image only when its device identifier matches

Table 2. Photo-submission API routes implemented in the source.

The learning pages do not expose a custom JSON API; they read the repository content through server-side application modules. The photo endpoints are the application's write-capable API surface.

3. Implementation

3.1. Project structure

The application follows the Next.js App Router structure. The main directories and files are:

```
ruatoan/
|-- app/                  # routes, pages, styles, and photo API
|   |-- bai/              # individual problem pages
|   |-- de/               # exam collection pages
|   |-- luyen/             # searchable practice list
|   |-- api/photos/        # photo submission endpoints
|   `-- ui/                # shared interface and browser components
|-- content/               # JSON problem, lesson, book, and document data
|-- lib/                   # content parsing, topics, and math rendering
|-- scripts/               # source collection and processing tools
|-- package.json
`-- vercel.json
```

3.2. User interface and problem rendering

Pages are composed from reusable React components for navigation, topic labels, collection cards, problem cards, and page headings. The problem statement is stored as lightweight Markdown with LaTeX delimiters. The renderer escapes ordinary HTML text and sends math expressions to KaTeX's server-side string renderer [1]. Topic aliases in older content are normalised into the five categories shown by the interface.

The exam page renders a full collection in order and includes a print control. A problem page provides previous/next navigation within its collection, related problems, and a submission form. On the client, photographs are resized, converted to grayscale, and compressed to WebP or JPEG before upload. The browser stores a compact progress summary and device token in local storage.

3.3. Photo storage and API logic

The API validates the device token, problem identifier, image MIME type, per-file size, number of images, and daily per-device count. It stores each image and its metadata in the PostgreSQL table `thcs_hard.photos`. Deletion checks that the request device matches the stored device hash. Image retrieval uses a UUID route. The browser does not send student account details because the application has no login system.

Listing 1. Device-scoped deletion of a submitted image.

```
export async function DELETE(req: Request, ctx: Ctx) {
  const dev = device(req);
  const { id } = await ctx.params;
  if (!dev || !UUID.test(id)) return json({ error: 'Khong hop le' }, 400);
  const sql = await db();
  const r = await sql`
    delete from thcs_hard.photos where id = ${id} and device = ${dev}`;
  return json({ ok: r.count > 0 });
}
```

Listing 1 illustrates the ownership check before deletion. The SQL template uses parameter interpolation for the supplied values.

3.4. Content collection and source control

The repository contains Python and Node.js scripts for collecting and preparing exam, magazine, olympiad, book, and document metadata. The website consumes the prepared JSON files rather than calling crawlers during a page request. Exam and problem pages display the stored source name and, when available, an outbound source URL.

The available project files do not document the team's branch, pull-request, or review workflow. The group should add the actual workflow and member contributions from its Git history and collaboration records rather than infer them from the application source.

3.5. Deployment

The repository contains a Vercel project configuration and selects the `sin1` (Singapore) region. The application metadata uses `https://ruatoan.vercel.app` as its default site URL. Photo upload requires the deployment environment to provide a PostgreSQL connection string; the credential itself is not included in this report. The configured hostname and production availability should be confirmed before submission.

4. Results and Conclusion

4.1. Verification status

The repository has no declared automated test command or test suite in its package scripts. The table below is therefore a verification checklist, not a claim that these cases have passed. Functional, responsive, database, and accessibility checks still need to be run on the deployed application and recorded with evidence.

ID	Area	Expected result	Status
TC01	Browse	Topic and exam filters return matching records	Not run
TC02	Math rendering	Inline and display expressions render on problem pages	Not run
TC03	Photo upload	Valid compressed images are saved and listed for the same device	Not run
TC04	Upload validation	Invalid type, oversized image, and excess daily count are rejected	Not run
TC05	Photo deletion	A device can delete its own image but cannot delete another device's image	Not run
TC06	Responsive layout	Navigation, filters, and problem text remain usable on a narrow screen	Not run
TC07	Print	Exam statement and problem order are readable in print output	Not run

Table 3. Functional verification checklist; results have not yet been measured.

No Lighthouse scores or browser-test results are reported because they were not collected for this report. Add dated screenshots or exported measurements after running the checks.

4.2. Achievements and limitations

The implementation provides a broad, structured mathematics library, five topic categories, exam collection pages, individual problem pages, mathematical notation rendering, related problem links, print support, and an account-free photo-submission flow. The repository currently contains 3,381 collections and 33,929 problem records, in addition to lessons, book metadata, and document records. These counts demonstrate the scale of the stored catalogue; they do not by themselves measure transcription accuracy or learning outcomes.

The product has several known limitations. Search and content loading operate over the JSON corpus at runtime, so response time and memory use may grow with the catalogue. There is no authenticated account, server-side progress synchronisation, automatic answer checking, or solution and hint feature. Photo submissions are

identified by a browser-held token, and the metadata endpoint returns at most 500 records at a time. Upload limits are checked in application code, and concurrent requests may require stronger quota enforcement. The current project also lacks recorded automated tests, accessibility results, and measured performance scores. The accuracy and completeness of collected problems require editorial review.

4.3. Lessons learned and future work

Separating content into structured records makes it possible to browse problems across multiple collections while retaining their source context. The project also illustrates that mathematical text needs a dedicated renderer and that image submissions require separate storage and lifecycle handling from the static learning catalogue.

Recommended next steps are:

- Add automated checks for content schema, broken links, duplicate records, and math rendering.
- Run and record functional, mobile, accessibility, and performance checks.
- Add pagination for photo history and make per-device upload quotas safe under concurrent requests.
- Move corpus indexing and expensive rendering work to a build step as the collection grows.
- Review source permissions, attribution, and transcription accuracy for every content source.
- Consider optional accounts and cross-device synchronisation only with a clear privacy and data-retention policy.

Rùa Toán has a clear educational focus and a working separation between its content catalogue and student work submissions. Completing the testing record, member details, and deployment evidence will make the report ready for academic submission.

References

[1] KaTeX Contributors. Katex api documentation, 2026. Accessed: 2026-11-01.

[2] PostgreSQL Global Development Group. Postgresql documentation, 2026. Accessed: 2026-11-01.

[3] Vercel. Next.js app router documentation, 2026. Accessed: 2026-11-01.

[4] Vercel. Next.js on vercel, 2026. Accessed: 2026-11-01.

A. Appendix

A.1. Setup and run instructions

```
git clone [repository URL]
cd ruatoan
npm install
npm run dev
```

The production scripts are `npm run build` and `npm start`. Local photo submission also requires a PostgreSQL connection string in the environment. Use the project's deployment settings or a development database; do not commit database credentials to the repository.

A.2. Demo accounts

The website has no account or login feature. Each browser creates a random device token for its submitted work, so there are no demo usernames or passwords.

A.3. Use of AI tools (if any)

ChatGPT was used to review the source code and draft this report from the repository. It did not run the application's test suite or independently verify the content catalogue. The group should add any other AI tools used during implementation and identify the work they assisted with, in accordance with the course requirements.